



Open in app ↗

Sign up

Sign in

Medium

🔍 Search

✍ Write



Generated by [Stable Diffusion](#)

Time Series Forecasting with XGBoost and LightGBM: Predicting Energy Consumption with Lag Features



George Kamtziridis

Follow

6 min read · Aug 6, 2023



164



5



In a previous [article](#), we've gone through the process of creating a model capable of predicting the demand of energy consumption for the city of London. Essentially, it was a time series forecasting problem where we had utilized the [London Energy Dataset](#) and the [London Weather Dataset](#) to build Ensemble models, such as XGBoost and LGBM, in order to accurately estimate the future needs of electric power. At first, we approached the task by modeling its *time-dependent* properties. Then, we added auxiliary information by incorporating weather data which improved the results by a significant margin. In this article, we will use the so-called *lag* features to boost the model performance even further.

The current article will use the [previous](#) one as a basis from which we will improve our models. I strongly advise you to meticulously read it before delving into this one, since we will be using its entire codebase.

Part 1: [Time Series Forecasting with XGBoost and LightGBM: Predicting Energy Consumption](#)

Part 3: Enhancing Time Series Forecasting with XGBoost: Incorporating Rolling Statistics and Moving Averages

Lag Features

As it is already mentioned, we've tackled the problem by exploring its *time-dependent* features. Even though these are generally considered to be the most influential in a time series scenario, there are additional properties that can be used to better model the task, like its *serially dependent* properties. To make use of these, one has to integrate past values of the target variable as input features. These past values are what we call *lag* features. For instance, in our problem we can use the demand of the previous day as an input feature while estimating the demand of the current day. Of course, we can include as many past days as we want where each will be considered as a separate lag feature.

Lag features are extremely useful for capturing *cycles*. In a time series, as cycles we define growths and decays in the target value that are not related to time, but mainly to previous target values. These fluctuations are not seasonal and their frequencies vary.

To take into account such cycles we need to use the lag features. To visualize the serial dependence we can use *lag plots*. One of the most popular lag plots is the autocorrelation plot, which showcases the correlation between the target and one specific lag.

To plot these we would need the following function:

```
1 # Make sure to import the required packages from the previous article too!
2
3 from matplotlib.offsetbox import AnchoredText
4 from statsmodels.graphics.tsaplots import plot_pacf
5 import seaborn as sns
6 import math
```

imports.py hosted with ❤ by GitHub

[view raw](#)

```

1  def plot_lag(x, lag=1, ax=None, **kwargs):
2      x_ = x.shift(lag)
3      y_ = x
4      if ax is None:
5          fig, ax = plt.subplots()
6          scatter_kws = dict(
7              alpha=0.75,
8              s=3,
9          )
10         line = dict(color='C3', )
11
12         ax = sns.regplot(x=x_,
13                         y=y_,
14                         scatter_kws=scatter_kws,
15                         line_kws=line,
16                         lowess=True,
17                         ax=ax,
18                         **kwargs)
19
20         # Adding correlation on plot
21         at = AnchoredText(
22             f'{y_.corr(x_):.2f}',
23             prop=dict(size="large"),
24             frameon=True,
25             loc="upper left",
26         )
27         at.patch.set_boxstyle("square, pad=0.0")
28         ax.add_artist(at)
29         ax.set(title=f"Lag {lag}", xlabel=x_.name, ylabel=y_.name)
30         return ax
31
32  def plot_autocorrelation(x, lags=6, lagplot_kwargs={}, **kwargs):
33      kwargs.setdefault("nrows", 2)
34      kwargs.setdefault("ncols", math.ceil(lags / 2))
35      kwargs.setdefault("figsize", (kwargs["ncols"] * 2, 2 * 2 + 0.5))
36      fig, axs = plt.subplots(sharex=True, sharey=True, squeeze=False, **kwargs)
37      for ax, k in zip(fig.get_axes(), range(2 * kwargs["ncols"])):
38          if k + 1 <= lags:
39              ax = plot_lag(x, lag=k + 1, ax=ax, **lagplot_kwargs)
40              ax.set_title(f"Lag #{k + 1}", fontdict=dict(fontsize=14))
41              ax.set(xlabel="", ylabel="")
42          else:
43              ax.axis("off")
44      plt.setp(axs[-1, :], xlabel=x.name)
45      fig.tight_layout(w_pad=0.1, h_pad=0.1)

```

```
45 fig.tight_layout(w_pad=0.1, h_pad=0.1)
46 return fig
```

plots.py hosted with ❤ by GitHub

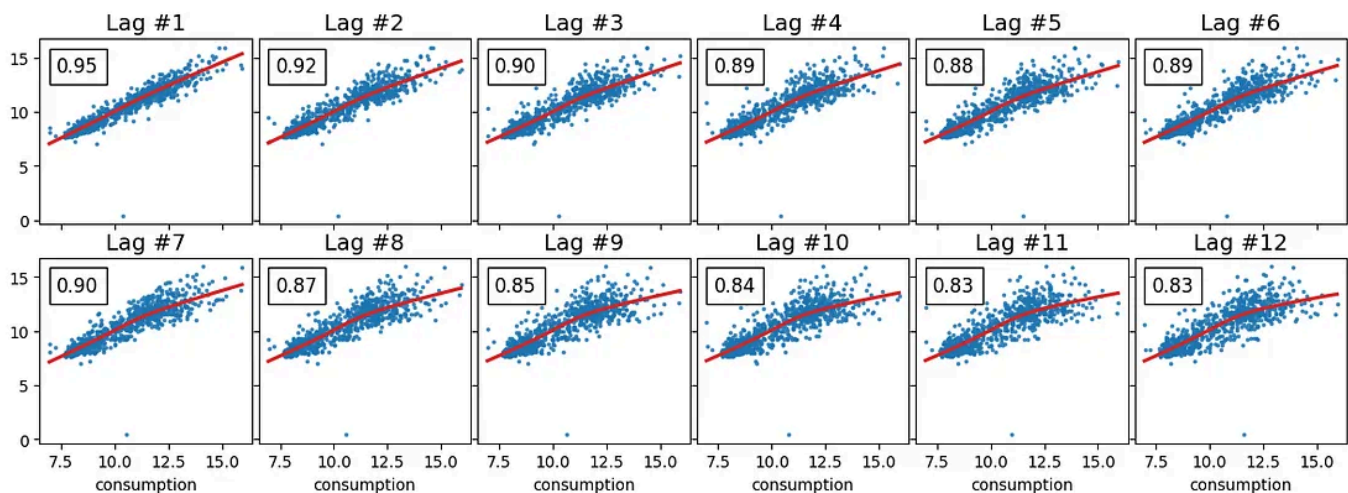
[view raw](#)

So, let's use the `df\_avg\_consumption` dataframe from the previous article to create the lag plots for 12 lags:

```
1 # Use the `df_avg_consumption` dataframe from the previous article.
2
3 _ = plot_autocorrelation(df_avg_consumption["consumption"], lags=12)
```

plot_autocorrelation.py hosted with ❤ by GitHub

[view raw](#)



Autocorrelation plots for 12 lag features

It is more than clear that there is a relationship, mostly linear, between the target and its lags, where the correlation drops by approximately 1–2% when moving from one lag to another. These findings indicate that we would have some gains in the model performance if we use some lag features.

Choosing Lag Features

So, the question is how do we know which lag features to include? In our scenario, should we choose 1 lag, 2 or 100? Most probably the previous day

will play some role in today's forecast, but what about the results of a week ago? Also, our intuition says that most likely lag 1 will contain some of the information contained in lag 2, while lag 2 will contain some of the information contained in lag 3 and so on. This means that we can end up having redundant lags even though their correlation with the target value might be high enough.

To properly choose our lags we need to check the partial autocorrelation plot. In a sense, partial autocorrelation plots outline the amount of “new” information coming from a lag by taking into account all the previous ones too. By doing so, we are able to determine which features are indeed useful and which hold information we already have.

Get George Kamtziridis's stories in your inbox

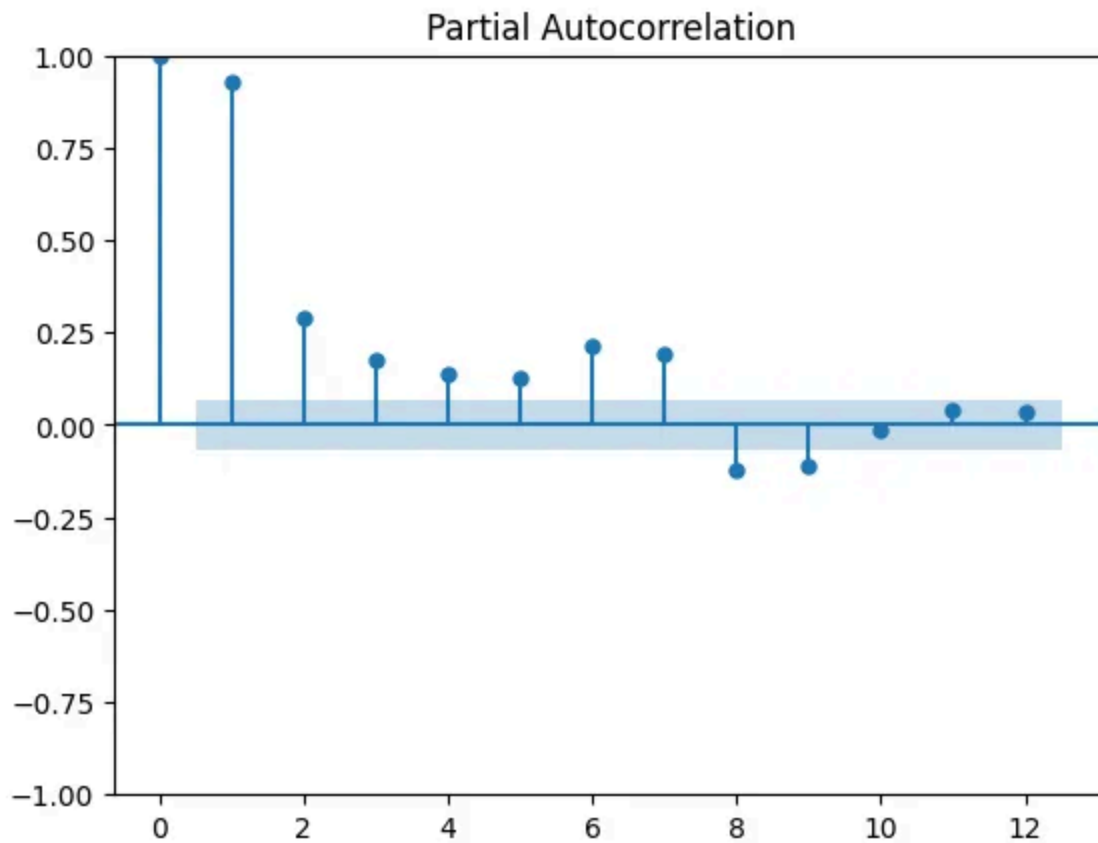
Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

We can plot these with the help of the `plot_pacf` function of the [statsmodels](#) Python package:

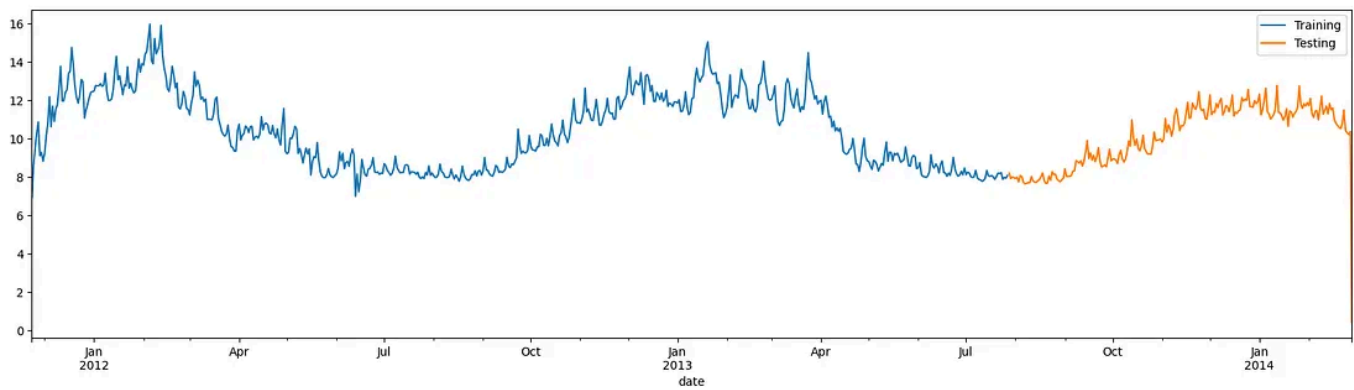


Partial autocorrelation plot for 12 lag features

We can clearly see that the first 9 lags possibly contain valuable information since they're out of the bluish area. This area is related to the confidence interval that was chosen in the `plot_pacf` function, with the default value being 95%.

Experiments

Before moving on to the experiments, let's quickly remember what's our task. As described in the previous [article](#), we want to forecast the energy consumption from August of 2013 to March of 2014 by training on data from November of 2011 to July of 2013.



Training and testing data

With that being said, we're going to run some experiments to check if the lag features actually improve our models, as well as if having more than 9 lags can lead to even more performant models.

We have run the following 3 experiments for each model:

- Using 4 lag features
- Using 9 lag features (partial autocorrelation plot marks this one as the best choice)
- Using 12 lag features

Of course, our baseline models will be the best models of the previous article. Their results are shown below:

To train the models we will use the codebase of the previous article with the only enhancement being the insertion of the lag features:

It should be noted that after adding the new lag features, the initial samples will contain NaN values. Generally speaking, the missing values should be filled-in somehow, but since we use XGBoost and LGBM we can leave them as is and our algorithms will take care of them.

The results of the 3 new experiments with XGBoost are:

XGBoost Results

It is fairly obvious that even the first experiment with only 4 lags significantly improves the model performance. The MAE is reduced by almost 35%, the MSE by 30.5% while the MAPE dropped from 16% to 13.4%.

In the second experiment, the results are even better. Incorporating 9 lags decreases the MAE by 35.7%, the MSE by 32% and, finally, the MAPE falls to 13.2%.

When utilizing 12 lags the model still performs better compared to the baseline, but the performance is worse than the previous 2 experiments. This is an indicator that extra lags not only fail to improve our results, but they might “confuse” the model leading to poorer performance.

Looking at the LGBM results, lags do improve the model too. However, the model reacts a bit differently when we add new lags compared to XGBoost. More specifically, it performs better when 4 lags are included with the performance dropping slightly when adding 9 or 12 lags.

In general terms, even though the partial autocorrelation plots provide decent information, one should play around with the different lag configurations before arriving to any concrete conclusion regarding which lags to choose.

Conclusion

Time series can have serially dependent properties too, such as cycles. One way of exposing these to our model is through the lag features. To properly choose the number of lag features one has to utilize lag plots, like autocorrelation and partial autocorrelation plots, as well as conducting extensive experiments. In our energy consumption forecasting scenario, using lag features improved our models remarkably, reducing MAE by 35% and MSE by 32%.

WRITER at MLearning.ai // Control AI Video 🤖 /imagine AI 3D Models

MLearning.ai Submission Suggestions

How to become a writer on Mlearning.ai

medium.com

Machine Learning

Data Science

Artificial Intelligence

Software Development

MI So Good



Written by George Kamtziridis

Follow

166 followers · 24 following

Lead AI / Software Engineer at fromScratch Studio. BEng, MEng (Electrical Engineering/Computer Engineering) MSc (Artificial Intelligence)
